# map pattern overview & optimization

## introduction

- the **Map Pattern** is one of the most fundamental and well-known patterns in parallel programming
  - often found in functional languages (like Haskell, Scala, ML)
- **core concept**
  - it applies a function to every element of a collection (or list) to produce a new collection of results
- **mathematical definition**
  - it transforms a list of type a into a list of type b using a specific function
    - $\forall a, b (a \rightarrow b)\, \text{list} < a > \rightarrow \text{list} < b >$

se applichiamo una funzione f a un array A, otteniamo un array B dove B[i]=f(A[i])

- **key requirement**
  - the function applied (the "Elemental Function") should be **pure**
    - ideally, f(x) should not depend on or modify global variables
    - if the function has side effects, changing the execution order (which parallelism does) changes the memory state unpredictably

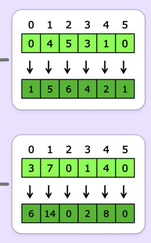
```
for (i=0; i<100; i++)
        for (j=i; j<100; j++)
          a[i][j] = f(a[i][j]);
```

  - the map function should not modify shared states
    - modifying shared state breaks perfect independence
  - if **independence** is broken (**ex.** thread A modifies a variable thread B reads), it leads to
    - non-determinism
    - data-races
    - undefined behavior
    - segfaults

## characteristics of Map

- **independence**
  - the defining feature of Map is that the operation on one element occurs without knowledge of its neighbors

è applicata ad un elemento senza bisogno di conoscere i valori degli elementi vicini (altrimenti sarebbe uno stencil)

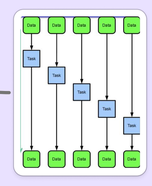



  - each iterations is completely independent of the others
    - **embarrassingly parallel:** because of this independence, Map can run completely in parallel with significant speedupds

non c'è comunicazione tra le iterazioni

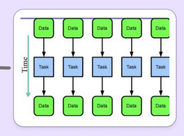
- **visualization**
  - **serial map**
    - a loop executes one task after another

```
for (int n=0; n< array.length; ++n){
  process(array[n]);
}
```




    - the total time is the sum of all task durations
  - **parallel map**
    - a construct (like parallel_for_each) executes tasks simultaneously across available processing units

```
parallel_for_each(x in array) {
  process(x);
}
```




    - with the parallel map, our program finished execution early, while the serial map is still running

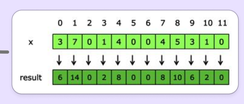
## types of Maps

- **unary map** → maps a function over a *single* collection

prende 1 collezione di input e produce 1 collezione di output

  - **input:** 1 collection
  - **output:** 1 collection
  - **ex.** double every item in an array

```
int oneToOne (int x[12]) {
  return x*2;
}
```

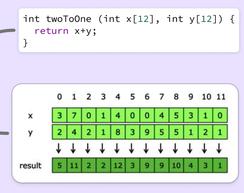



- **n-aray map** → maps a function over *multiple* collections simultaneously

prende N collezioni di input (solitamente alla stessa lunghezza) e produce 1 collezione di output

  - **input:** N collections
  - **output:** 1 collection
  - **ex.** *vector addition* → takes element i from Array X and element i from Array Y to produce Result i

```
int twoToOne (int x[12], int y[12]) {
  return x+y;
}
```




## implementation and API

- different parallel system implement Map using different syntaxes
  - **OpenMP & Cilk Plus**
    - use parallel for constructs
  - **TBB (Threading Building Blocks)**
    - use higher-order functions with lambda expressions
  - **array notation (CilkPlus, Fortran, Matlab)**
    - concise syntax like A[:] = A[i] * 5
      - this is the notation for "*multiply every element in A by 5*"
- le operazioni che stiamo facendo avvengono su tutti gli elementi
  - CUDA è diverso perché prova a descrivere il kernel come se fosse un tipo specifico di thread

## optimizations: sequence of maps

- in many algorithms (**ex.** vector math), operations occur in a sequence: first add, then multiply, then substract
  - **problem:** a naive implementation executes these as separate Map passes
    - read memory → add → write memory
    - read memory → multiply → write memory
  - **cost:** this approach wastes *memory bandwidth* by repeatedly writing and reading intermediate results to/ from main memory, potentially overwhelming the cache

**Il Problema (Implementazione Naive):** Se eseguiamo prima la Map 1 su tutto l'array, scriviamo i risultati in memoria (DRAM). Poi la Map 2 deve rileggere quei risultati dalla memoria. Questo spreca **Banda di Memoria** (Memory Bandwidth) e inquina la cache .

example

```
for (int i = 0; i < n; ++i) {
    a[i] = a[i] * 2;
}
for (int i = 0; i < n; ++i) {
    a[i] = a[i] + 3;
}
```

- to solve this, two primary optimization patterns are used
  - **code fusion**
    - the **idea** is to "fuse" the operations together into a single function

invece di fare due passaggi separati sulla memoria, si "fondono" le operazioni matematiche in una singola funzione elementare più complessa



    - **mechanism**
      - instead of two loops, we write one loop that performs both the addition and multiplication on an element before moving to the next
    - **benefit**
      - this increases **arithmetic intensity** (more math per byte loaded)

l'intensità aritmetica aumenta perché abbiamo più calcoli per ogni byte letto dalla memoria

      - ideally, intermediate data stays in **registers**, never touching main memory
  - **cache fusion**
    - the **idea** is to organize the *execution* to keep data in the cache
      - it is useful when code cannot be fused because perhaps they are separate library modules

possiamo cambiare l'ordine di esecuzione per cui invece di processare tutto l'array orizzontalmente, dividiamo il lavoro in blocchi (tile) che stanno nella cache

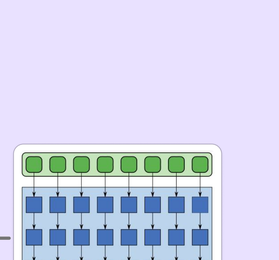




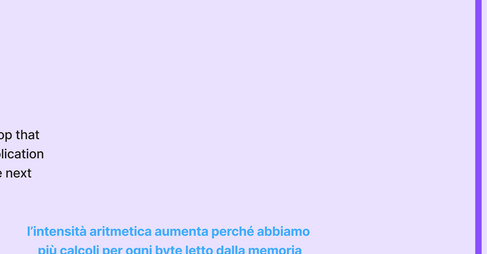

    - **mechanism**
      - break the work into **blocks**
        - the CPU processes all operations (Map 1, Map 2, etc) on Block 1 while it sits in the cache, before moving to Block 2
      - in questo caso, andiamo a lavorare su un sottoinsieme di dati su cui completiamo tutte le operazioni prima di passare al secondo layer
        - lavoriamo in direzione verticale invece che in direzione orizzontale
    - **benefit**
      - reduced cache misses and memory traffic, even if the code logic remains separate

## related patterns

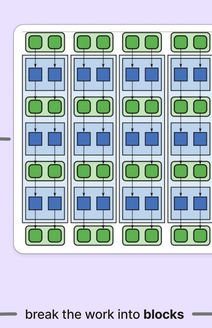
- several patterns are closely related to Map but differ in specific constraints
  - **stencil**
    - like Map, but the elemental function accesses neighbors of the input



      - proviamo a fare una convoluzione
      - proviamo a minimizzare gli accessi alla memoria (nel caso di CUDA udavamo la shared memroy)
      - **ex.** A[i] needs A[i-1]



    - common in image processing (blurring) and PDE solvers
  - **workpile**
    - a generalized Map where items can be **added** to the workload *while* the map is running
      - the "pile" of work grows and is consumed dynamically
      - terminates when the pile is empty
  - **divide-and-conquer**
    - recursively divides a problem into smaller subproblems until a base case is reached



    - used when the problem structure implies a recursive split (like sorting) rather than a flat iteration over an array